



Uplift Modelling via Gradient Boosting

Bulat Ibragimov*

ibrbulat@yandex.ru

Moscow Institute of Physics and Technology
Moscow, Russian Federation
Artificial Intelligence Research Institute (AIRI)
Moscow, Russian Federation

Anton Vakhrushev*

btpanda@gmail.com

Sber AI Lab
Moscow, Russian Federation

ABSTRACT

The Gradient Boosting machine learning ensemble algorithm, well-known for its proficiency and superior performance in intricate machine learning tasks, has encountered limited success in the realm of uplift modeling. Uplift modeling is a challenging task that necessitates a known target for the precise computation of the training gradient. The prevailing two-model strategies, which separately model treatment and control outcomes, are encumbered with limitations as they fail to directly tackle the uplift problem.

This paper presents an innovative approach to uplift modeling that employs Gradient Boosting. Unlike previous works, our algorithm utilizes multioutput boosting model and calculates the uplift gradient based on intermediate surrogate predictions and directly models the concealed target. This method circumvents the requirement for a known target and addresses the uplift problem more effectively than existing solutions.

Moreover, we broaden the scope of this solution to encompass multitreatment settings, thereby enhancing its applicability. This novel approach not only overcomes the limitations of the traditional two-model strategies but also paves the way for more effective and efficient uplift modeling using Gradient Boosting.

CCS CONCEPTS

• **Computing methodologies** → **Causal reasoning and diagnostics; Boosting.**

KEYWORDS

Causal Inference, Uplift, Boosting

ACM Reference Format:

Bulat Ibragimov and Anton Vakhrushev. 2024. Uplift Modelling via Gradient Boosting. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*, August 25–29, 2024, Barcelona, Spain. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3637528.3672019>

1 INTRODUCTION

Uplift modeling, as defined by Gutierrez et al. [12], is a method used to determine the incremental impact of a specific action or treatment on a customer's outcome. This technique is particularly

useful in a variety of scenarios. For instance, a marketplace might be interested in quantifying the effect of sending a promotional email to various customer profiles, offering a personal discount on certain types of goods. The impact of such an action can vary greatly among customers. Some might be annoyed by the email, while others might have already planned to make the purchase, and the discount offer merely reduces the potential profit. Therefore, uplift modeling is not just a Machine Learning problem, but also a Causal Inference one, as it requires estimating the difference between two mutually exclusive outcomes for an individual (i.e., whether a person receives a promotional email or not).

However, a naive approach to uplift modeling, which involves fitting two separate models (one for the control group and another for the treated group) and subtracting their outcome predictions, is fraught with issues [24]. These issues include a large magnitude of outcome compared to the uplift value, a noisy result due to the independence of the models, and a discrepancy between the features important for outcome prediction and those impacting the uplift. As a result, more sophisticated approaches that aim to model the uplift directly have been proposed in recent years [1, 18, 22]. Some of these approaches include tree-based methods. However, to the best of our knowledge, **Gradient Boosting has not been considered in these algorithms, primarily because the gradient for uplift is not well-defined since every data instance belongs to either the control or treated group.** Another issue is that the **proposed models are not adapted to a multitreatment setting, where the size of the allowed treatments set is more than one.**

One potential algorithm that could be applied to the uplift modeling problem is Gradient Boosting. As introduced by Friedman [10], **Gradient Boosting** is a powerful and modern algorithm that achieves **state-of-the-art results on tabular data**, with categorical features, noisy features and labels, and missing data [3, 20, 31]. Another reason for the popularity of Gradient Boosting is its relatively **low time and memory costs** compared to today's other widely used models, such as neural networks [7]. This is why, despite the dramatic growth of general-purpose neural models, Gradient Boosting is still used in many areas (such as ranking [4], recommender systems [6], automatic machine learning [19], and many other tasks [21, 28, 29]) and continues to find new applications.

In this paper, we introduce a novel approach to uplift modeling that leverages the power of Gradient Boosting and extends its application to **multitreatment models**. Our method **builds upon the existing Gradient Boosting implementation, known as PyBoost, and adapts it to address causal inference problems.** We delve into the exploration of a more effective solution for uplift modeling that surpasses the limitations of current methods.

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Our approach specifically addresses the challenge of fitting separate models for treatment and control outcomes. We propose a unified model that leverages a multi-output regression Gradient Boosting algorithm. This model predicts both treatment and control outcomes simultaneously, thereby eliminating the need for separate models and reducing variance in predictions. Furthermore, we introduce a secondary boosting model that refines these predictions by focusing on the surrogate uplift target, enhancing the robustness and interpretability of the uplift estimates.

The primary model in our system is a multioutput regression (or classification) Gradient Boosting model. It utilizes the same trees for predicting every treatment and control outputs. This uniformity results in the same structure for the trained piecewise-constant functions, leading to more stable uplift predictions.

The secondary model plays a crucial role in our approach. It is designed to find simpler surfaces that explain the uplift dependence on features. This simplicity can be advantageous for exploratory data analysis, leading to more robust predictions. Furthermore, the secondary model allows for the use of early stopping techniques. These techniques are based on the quality estimators of uplift metrics for the secondary tree, providing an additional layer of efficiency to our approach.

In summary, our novel approach to uplift modeling using Gradient Boosting, including multitreatment models, offers a more effective and efficient solution. By extending the existing Gradient Boosting implementation and introducing a secondary model, we are able to address the limitations of current methods and provide a more robust solution in terms of MSE and AUUC (area-under-uplift-curve) metrics on various datasets.

The rest of this paper is organized as follows: Section 2 provides background information and reviews related work in the field of uplift modeling and Gradient Boosting. Section 3 describes our proposed method in detail, including the novel approach to handling multitreatment uplift modeling. Section 4 presents the results of our numerical experiments, demonstrating the effectiveness of our approach on real-world and synthetic datasets. Finally, Section 5 concludes the paper with a summary of our findings and suggestions for future research directions.

2 BACKGROUND AND RELATED WORK

2.1 Gradient Boosting

Given a dataset $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^n$, where $\mathbf{x}_i = (x_i^1, \dots, x_i^m) \in \mathbb{X}$ is an m -dimensional feature representation and $y_i \in \mathbb{Y}$ is a target value of the i -th observation, the learning algorithm aims to find a function $F : \mathbb{X} \rightarrow \mathbb{Y}$ minimizing the expected target prediction error. Prediction error estimator is defined via introducing some differentiable by the first argument loss function $L : \mathbb{Y} \times \mathbb{Y} \rightarrow \mathbb{R}_+$:

$$\mathcal{L}(\mathcal{D}, F) := \sum_{i=1}^n [L(F(\mathbf{x}_i), y_i)] \rightarrow \min_F.$$

Gradient Boosting (GB) algorithm builds the desired function F as a weighted sum (or an ensemble) of functions from the set $\mathcal{F}$ of, usually, weak learners (e.g. shallow decision trees or neural networks). So the final model is an ensemble of B base functions $\{f_1, f_2, \dots, f_B\}$:

$$F_B(\mathbf{x}) = \sum_{i=1}^B \alpha_i f_i(\mathbf{x}),$$

where $\{\alpha_i\}_{i=1}^B$ are the weights, which are usually equal to the same value α that is a hyperparameter of the model.

The approximate solution of the latter equation in GB is usually constructed as follows. Algorithm calculates first and second order derivatives of $\mathcal{L}$ at the point F_{t-1} w.r.t. predicted values:

$$g_i^t = \left. \frac{\partial L(\hat{y}_i, y_i)}{\partial \hat{y}_i} \right|_{\hat{y}_i = F_{t-1}(\mathbf{x}_i)}, \quad h_i^t = \left. \frac{\partial^2 L(\hat{y}_i, y_i)}{\partial \hat{y}_i^2} \right|_{\hat{y}_i = F_{t-1}(\mathbf{x}_i)},$$

and selects a least squares estimator to Newton's gradient step in the functional space:

$$f_t = \arg \min_{f \in \mathcal{F}} \sum_{i=1}^n h_i^t(\mathbf{x}_i, y_i) \left(f(\mathbf{x}_i) - \left(-\frac{g_i^t(\mathbf{x}_i, y_i)}{h_i^t(\mathbf{x}_i, y_i)} \right) \right)^2,$$

see [5] for details.

The ability to calculate the derivative of the error with respect to the current prediction is a fundamental requirement for the algorithm, as demonstrated in the previous derivations.

2.2 Uplift Modelling

Uplift modeling is a predictive modeling technique that assesses the incremental impact of a treatment (such as a marketing campaign) on various individual outcomes. These outcomes can include, but are not limited to, behavioral changes, purchasing decisions, and other measurable traits of interest. This technique finds applications in various fields, including customer targeting, political campaigns, and personalized medicine.

Uplift modeling has the potential to significantly improve marketing efficiency and effectiveness by targeting the individuals who are most likely to have their behavior influenced by the treatment.

Formally, let's denote the treatment indicator as W , where $W = 1$ if treated and $W = 0$ otherwise. The potential outcome of individual i under treatment w is denoted as $Y_i(w)$. However, for each individual, we can only observe one of the potential outcomes, $Y_i = Y_i(W)$, because an individual can either be treated or not treated, but not both.

The uplift for individual i is defined as the difference in potential outcomes under treatment and control:

$$U_i = Y_i(1) - Y_i(0)$$

However, since we can't observe both $Y_i(1)$ and $Y_i(0)$ for the same individual, we can't directly measure U_i . Instead, uplift modeling aims to estimate the expected uplift conditional on individual feature representation X_i :

$$U(X_i) = E[U_i | X_i] = E[Y_i(1) - Y_i(0) | X_i]$$

The goal of uplift modeling is to estimate the function $U(X)$ based on observed data, and use this estimate to predict the uplift of future individuals based on their covariates. This allows us to target individuals with the highest predicted uplift.

It's important to note that uplift modeling is fundamentally different from traditional predictive modeling, which aims to predict

the outcome Y_i rather than the uplift U_i . Therefore, traditional model evaluation metrics like accuracy are not appropriate for uplift models. Instead, uplift models are typically evaluated using uplift curves or Qini curves, which measure the incremental impact of the treatment at different levels of targeting.

In conclusion, uplift modeling is a powerful tool for optimizing the impact of treatments in fields like marketing, healthcare, and public policy. However, it also poses unique methodological challenges and requires careful consideration of the underlying assumptions and evaluation metrics.

2.3 Uplift Modelling Methods

In practice, uplift modeling can be approached in several ways, including two-model approach, single-model approach, and tree-based methods. Each approach has its own strengths and weaknesses, and the choice of approach depends on the specific application and available data.

Decision trees have gained substantial popularity as a base model for uplift modeling. Their interpretability and the ability to incorporate specific properties into the splitting criteria make them a preferred choice. These properties can include the differential effect of treatment and control or the regularization of imbalance in the leaves. Various methodologies for constructing uplift trees have been proposed in the literature. Some notable ones include the $\Delta\Delta p$ method [13], the use of t-statistics [27], and the maximum likelihood estimation of uplift score [26]. Each of these methods brings unique contributions to uplift modeling. Typically, these methods resort to building an ensemble of trees using a technique known as bagging to achieve improved and more stable results.

Another approach to use decision tree ensembles is the Causal Forest model [30], a variant of the Random Forest algorithm, specifically designed for estimating heterogeneous treatment effects. It predicts individual treatment effects by building a forest of decision trees, each maximizing the difference in treatment effects between child nodes. The treatment effect for a new observation is estimated by averaging the treatment effects of similar observations across the forest.

In recent years, meta-learners have emerged as a popular approach in uplift modeling. A meta-learner is a framework that employs any learning algorithm (typically an ensemble) to estimate the difference in average outcomes for a treated individual and a control one. This difference is referred to as the conditional average treatment effect (CATE). The most straightforward approach is the T-learner [18], which trains two separate models (for treatment and control groups) and then calculates the difference between predictions for the test set. Another similar approach is the S-learner [2, 11, 14], which uses a single model with additional binary features indicating treatment or control group.

The X-learner [18] is a three-stage process. Initially, it estimates the response functions under control and treatment using base learners. Subsequently, using the estimated response functions, it imputes the individual treatment effects for both groups. Finally, it estimates the CATE function by a weighted average of two estimators based on the imputed treatment effects (usually with propensity scores as weights).

The R-learner [22] operates on a key idea of first estimating marginal effects and treatment propensities using a technique known as cross-fitting. These estimates are then used to form an objective function that isolates the causal component of the signal. The second step involves optimizing this objective function using any machine learning method.

The paper by Kennedy et al. [17] proposes a doubly robust CATE estimator, referred to as the DR-learner. The algorithm divides a dataset into two parts, trains models for predicting treatment and control outcomes and propensity scores, constructs a pseudo-outcome for the second part of the dataset, and trains another (final) model.

Another interesting approach is described in the work by Devriendt et al. [8], where the authors investigate different ways to formulate uplift modeling as a learning to rank problem. They introduce a new metric for learning to rank that captures the area under the uplift curve, called the promoted cumulative gain (PCG). They also demonstrate how PCG can be integrated into the LambdaMART algorithm, a state-of-the-art listwise learning to rank technique. This approach opens up new possibilities for uplift modeling by leveraging techniques from the field of learning to rank.

Deep Embedded Self-Controlled Network (DESCN) [32] is another state-of-the-art method specifically designed for uplift modeling. DESCN employs deep learning techniques to simultaneously learn representations and predict treatment effects. This model uses a shared network to learn common representations of the input features, followed by separate subnetworks for each treatment group to capture the treatment-specific effects.

Another notable uplift deep learning model is Dragonnet [25]. It operates in two stages: first, it fits models for the expected outcome and the probability of treatment (propensity score) for each unit. Then these fitted models are used in a downstream estimator of the effect. It also incorporates a regularization procedure, targeted regularization, that induces a bias towards models that have non-parametrically optimal asymptotic properties.

3 UPLIFT MODELLING VIA GRADIENT BOOSTING

Uplift modeling involves complex relationships between features and often lacks known target data. Gradient Boosting, known for capturing nonlinear dependencies, is a flexible technique suitable for this task.

Most uplift modeling algorithms predict uplift scores by subtracting control predictions from treatment predictions. This can be suboptimal due to varying feature importances, different outcome magnitudes, and independent model variances.

Another critical issue when constructing a boosting model is the unknown uplift target, making the evaluation of a training gradient for base learners a challenging task.

We propose a novel approach using a multi-output Gradient Boosting model to predict treatment and control outcomes simultaneously. This method provides consistent and stable predictions, improving reliability. We then refine these predictions using a secondary boosting model focused on the surrogate uplift target. This two-step process enhances prediction quality and easily adapts to multitreatment settings without compromising performance.

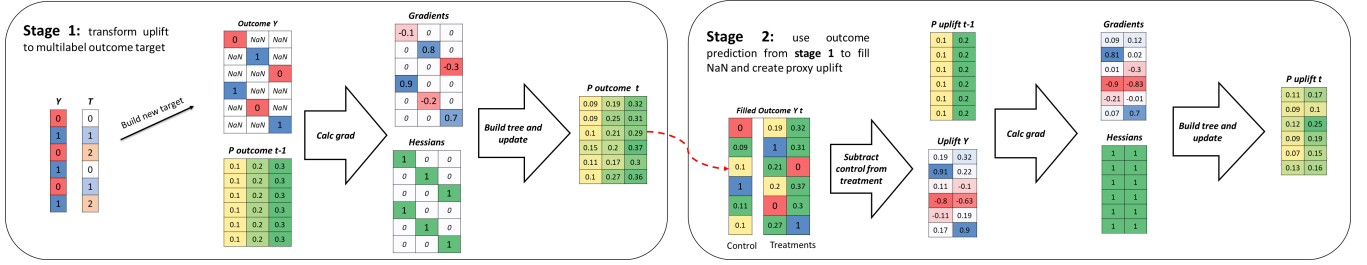


Figure 1: Two staged boosting algorithm.

In the first stage, we convert the multitreatment uplift task into multilabel tasks, incorporating unknown values within the target matrix. To manage these unknowns, we employ a masking technique during gradient computation, substituting them with zeros. In the second stage, we use the predictions from the first stage to impute unknown values and obtain a proxy uplift target. This pipeline represents a single boosting step, where both ensembles learn simultaneously.

3.1 Uplift Modelling as Multioutput Problem

We start by defining the base algorithm, which is designed to predict both the treatment outcome and the control outcome within a single boosting model. We employ a multioutput **Gradient Boosting algorithm, coupled with a single-tree strategy, to achieve a high-quality model**. This approach helps us reduce the computational complexity and the variance that arises due to the independence of predictions. In this context, the correlation of predictions refers to the fact that the base model trains a single multioutput tree at each boosting iteration. Consequently, there is no need to construct several distinct models for treatment and control; all predictions are made by the general ensemble. This results in a model that is more efficient in terms of space and time, less susceptible to overfitting, and can be easily scaled to accommodate multitreatment cases (as discussed in Section 3.3).

Formally speaking, for each individual i , the target variable and the predicted value are represented as two-dimensional vectors, $Y_i = (Y_i(0), Y_i(1))$ and $\hat{Y}_i = (\hat{Y}_i(0), \hat{Y}_i(1))$, respectively. Given this setup, the gradient for individual i after t boosting rounds is a two-dimensional vector:

$$g_i^t = \left(\frac{\partial L(\hat{Y}_i, Y_i)}{\partial \hat{Y}_{i0}} \Big|_{\hat{Y}_i = F_{t-1}(X_i)}, \frac{\partial L(\hat{Y}_i, Y_i)}{\partial \hat{Y}_{i1}} \Big|_{\hat{Y}_i = F_{t-1}(X_i)} \right).$$

For second-order methods, it is common practice to retain only the diagonal elements of the Hessian matrix to avoid the computational burden of matrix inversion. Therefore, we can assume

$$h_i^t = \left(\frac{\partial^2 L(\hat{Y}_i, Y_i)}{\partial \hat{Y}_{i0}^2} \Big|_{\hat{Y}_i = F_{t-1}(X_i)}, \frac{\partial^2 L(\hat{Y}_i, Y_i)}{\partial \hat{Y}_{i1}^2} \Big|_{\hat{Y}_i = F_{t-1}(X_i)} \right).$$

The optimal value for output j in leaf L is given by the following formula $o_{Lj}^t = -\frac{\sum_{i \in L} g_{ij}^t}{\sum_{i \in L} h_{ij}^t}$ (for simplicity, we omit regularization terms which are beyond the scope of this paper). Ultimately, the tree splitting criterion seeks to maximize the score

$$S(L, R) = \sum_{i \in \{L, R\}} \sum_{j=0}^1 \frac{\left(\sum_{i \in L} g_{ij}^t \right)^2}{\sum_{i \in L} h_{ij}^t}, \quad (1)$$

where L and R represent the left and right sides of the split, respectively.

The model described above is capable of modeling uplift values. This can be achieved by subtracting o_{L0}^t from o_{L1}^t for each tree t and leaf L . Or, in other terms:

$$\hat{U}(X_k) = \hat{Y}_k(1) - \hat{Y}_k(0) = \sum_{i=1}^B \alpha_i (f_i(X_k)(1) - f_i(X_k)(0))$$

As previously articulated, this method boasts several advantages, including scalability, efficiency, and a diminished propensity to overfit. The benefits derived from employing a single-tree multi-output boosting approach may stem from the fact that if one trains two separate trees at each iteration, they end up with two distinct piecewise-constant functions. This implies that the uplift prediction obtained for the X_k is computed by subtracting predictions aggregated from intersecting but different regions, which can introduce additional noise to the predictions. Specifically, the mean squared error of uplift predictions can be reformulated as follows:

$$\begin{aligned} \mathbb{E}_X \left(\hat{Y}(1) - \hat{Y}(0) - Y(1) + Y(0) \right)^2 = \\ \mathbb{E}_X \left(\hat{Y}(1) - Y(1) \right)^2 + \mathbb{E}_X \left(\hat{Y}(1) - Y(1) \right)^2 - \\ 2\mathbb{E}_X \left(Y(1) - \hat{Y}(1) \right) \left(Y(0) - \hat{Y}(0) \right). \end{aligned} \quad (2)$$

The first two terms in this equation constitute the loss of treatment and control outcome predictions, which can be directly optimized by the model.

It's important to recall that predictions are derived from a piecewise constant function. This means that there exists a partitioning of the feature space $\mathcal{A} = \{A_1, A_2, \dots, A_P\} : \bigcup A_i = \mathbb{X}$, such that for all i and for all $x, x' \in A_i : F_B(x) = F_B(x')$. Considering the following assumptions, which are reasonable in the context of uplift modelling: 1) The model yields unbiased (or small bias) predictions; 2) The random variables $Y(1)$ and $Y(0)$ are positively correlated. So for the third term of the Equation 2 we have:

$$\begin{aligned}
& \mathbb{E}_X \left(Y(1) - \hat{Y}(1) \right) \left(Y(0) - \hat{Y}(0) \right) = \\
& \mathbb{E}_{A \sim \mathcal{A}} \mathbb{E}_{X \sim A} \left(Y(1) - \hat{Y}(1) \right) \left(Y(0) - \hat{Y}(0) \right) \approx \\
& \mathbb{E}_{A \sim \mathcal{A}} \mathbb{E}_{X \sim A} [Y(1) - \mathbb{E}(Y(1)|A)] [Y(0) - \mathbb{E}(Y(0)|A)] = \\
& \text{cov}_X(Y(1), Y(0)) > 0,
\end{aligned}$$

which, according to Equation 2, reduces the mean squared error. As we can see, this does not necessarily hold for models where treatment and control outcomes are predicted independently, since they are obtained by averaging from potentially different regions.

However, this base model also presents potential drawbacks. Firstly, it does not directly model uplift; instead, it aims to fit treatment and control outcomes, which may result in some loss of information from the data. Secondly, the interpretation of the model can be challenging. Each tree tends to find splits that best explain the values of treatment and control outcomes, but there is no explicit indication of how the decision on the current prediction was made. We will address these limitations in the subsequent section.

3.2 Surrogate Target for Gradient Boosting

In the realm of uplift modeling, the actual target remains elusive. The primary objective of uplift modeling is to estimate the difference in outcomes between a group that has been treated and a control group. The "true" uplift remains a mystery because we can observe the outcome under treatment or control for any given individual, but not both simultaneously. This presents a significant challenge as it renders the calculation of the appropriate gradient needed for Gradient Boosting unattainable.

The predictions procured from the multioutput model, while reasonable, lack interpretability and are obtained as a superposition of two outcome predictions. If a more straightforward dependence can yield the same or similar predictions, identifying it will aid in enhancing the robustness and interpretability of the model. This line of thought leads us to the learning of an additional Gradient Boosting model, which takes the predictions from the base model as ground truth and aims to approximate them. A simpler model that approximates the predictions of a complex multioutput model could be easier to interpret, and if the secondary model can capture the essential patterns in the data, it might exhibit more robustness to noise and outliers in the data.

To formalize this, the target variable for the k -th object at step b is defined as

$$U_k^b = \begin{cases} Y_k(1) - \hat{Y}_k^b(0) = Y_k(1) - \sum_{i=1}^b \alpha_i f_i(X_k)(0), & \text{if } W_k = 1 \\ \hat{Y}_k^b(1) - Y_k(0) = \sum_{i=1}^b \alpha_i f_i(X_k)(1) - Y_k(0), & \text{otherwise} \end{cases}$$

and this is the sole deviation from the standard Gradient Boosting training protocol. To ensure that this model converges to a good uplift prediction, we employ the AUUC metric as a validation score.

To address concerns about potential contamination of the second-stage model due to erroneous predictions from the first-stage model,

we base our approach on several assumptions. We assume that sufficient information about the treatment and control groups is present in the training data, making the first model learnable. Additionally, due to Equation 2 and the Cauchy–Schwarz inequality, the fitting of treatment and control outcomes helps to bound the uplift error. These theoretical foundations suggest that our pseudo-labels for the second step are more reliable than random noise. This assumption aligns with classic learnability results [9, 16].

Since this secondary model approximates the base model, there might exist a trade-off between simplicity and accuracy. It is crucial to ensure that the simplification does not significantly compromise the predictive performance. To handle this, we employ a mixture of base and secondary models as the final forecaster. The mixture weights are responsible for managing the trade-off between a more straightforward and robust secondary model and a more accurate base model. Moreover, if the weight is tuned with respect to a separate validation set, it allows us to properly regularize the base model and provides evidence of how closely the second model aligns with the required uplift predictions. The latter also means that the secondary model in some cases may directly tune the number of boosting steps by utilizing early stopping methods on uplift metrics.

The training process of this two-stage uplift boosting model is visualized in Figure 1.

3.3 Multitreatment Case

The concept of multitreatment cases introduces an additional layer of complexity. Multitreatment cases refer to scenarios where there are more than two possible treatments that an individual can receive. In such cases, the uplift model aims to estimate the difference in outcomes between each treatment and a control group.

The method previously described can be extended to handle multitreatment cases by leveraging the power of multioutput boosting. The idea is indeed straightforward: use multioutput boosting for predicting different treatments. Here's how it works:

1. **Defining the Problem:** In multitreatment cases, the treatment indicator W can take on multiple values, each representing a different treatment. For instance, $W = 0$ could represent the control group, $W = 1$ could represent treatment 1, $W = 2$ could represent treatment 2, and so on. The potential outcome of individual i under treatment w is denoted as $Y_i(w)$.

2. **Multioutput Boosting:** In this context, multioutput boosting is used to predict the potential outcomes under each treatment. The target variable and predicted value for individual i are now multidimensional vectors, with each dimension corresponding to a different treatment. For example, $Y_i = (Y_i(0), Y_i(1), Y_i(2), \dots)$ and $\hat{Y}_i = (\hat{Y}_i(0), \hat{Y}_i(1), \hat{Y}_i(2), \dots)$ respectively.

3. **Gradient Calculation:** The gradient for individual i after t boosting rounds is also a multidimensional vector, with each dimension corresponding to the partial derivative of the loss function with respect to the predicted value under each treatment.

4. **Model Training:** The multioutput boosting model is trained using the same protocol as in the binary treatment case, but now taking into account multiple treatments. The optimal value for output j in leaf L and the tree splitting criterion are calculated in the same way, but now for each treatment.

5. Uplift Calculation: The uplift for each treatment is calculated by subtracting the predicted outcome under the control group from the predicted outcome under each treatment. For example, the uplift for treatment 1 is $\hat{U}_i(1) = \hat{Y}_i(1) - \hat{Y}_i(0)$, the uplift for treatment 2 is $\hat{U}_i(2) = \hat{Y}_i(2) - \hat{Y}_i(0)$, and so on.

6. Model Interpretation: As with the binary treatment case, interpreting the model can be challenging due to the complex dependencies between features and treatments. However, the use of multioutput boosting allows for a more straightforward interpretation of the model, as it directly models the uplift for each treatment.

In conclusion, the extension of the method to multitreatment cases involves the use of multioutput boosting to predict the potential outcomes under each treatment, and the calculation of uplift for each treatment. This approach maintains the advantages of scalability, efficiency, and reduced overfitting, while also providing a more straightforward interpretation of the model in the context of multiple treatments.

4 NUMERICAL EXPERIMENTS

4.1 Implementation details

The proposed approach was implemented as a two-staged gradient boosting algorithm. During the training procedure, it simultaneously fits two independent model components.

- **Outcome gradient boosting** is designed to predict $\hat{Y}$ for the control group and each treatment group. It can be utilized independently to forecast uplift as $\hat{Y}(t) - \hat{Y}(0)$ for treatment t . The loss function employed to compute gradients is determined by the initial task; for instance, in the case of binary outcomes, it could be cross-entropy or mean squared error for contiguous treatments.
- **Uplift gradient boosting** is focused on predicting $\hat{U}$ for each treatment group. The primary challenge in obtaining gradient values for these trees is the unavailability of real uplift values. Hence, estimates from the **Outcome** component $\hat{Y}^b(t)$ and $\hat{Y}^b(0)$ are employed to approximate the targets U^b , facilitating the estimation of gradients at boosting step b .

Both components can be independently utilized to estimate uplift. However, we determine the final prediction as a weighted average between the **Outcome** and the **Uplift** outputs, $w\hat{U} + (1-w)(\hat{Y}(t) - \hat{Y}(0))$. The detailed algorithm for constructing the ensembles is provided in the Appendix as Algorithm 1.

Gradients of outcome tree. To fit multioutput trees for the **outcome** ensemble, adjustments to the gradient function computations are necessary. In the case of binary treatments, where we represent the targets as $Y_i = (Y_i(0), Y_i(1))$, we only observe one of these values in reality, and making assumptions about the unseen value is not feasible. Therefore, instead of filling the targets, we will modify the gradient and Hessian values for treatment t as follows:

$$g_i(t) = \begin{cases} \frac{\partial L(\hat{Y}_i(t), Y_i(t))}{\partial \hat{Y}_i(t)}, & \text{if } Y_i(t) \text{ is observed} \\ 0, & \text{otherwise} \end{cases}$$

$$h_i(t) = \begin{cases} \frac{\partial^2 L(\hat{Y}_i(t), Y_i(t))}{\partial^2 \hat{Y}_i(t)}, & \text{if } Y_i(t) \text{ is observed} \\ 0, & \text{otherwise} \end{cases}$$

This modification is justified by the necessity to disregard unseen values when calculating both the split criteria and leaf value estimations. By masking these values, we can utilize these gradients and Hessians in Equation 1, since it would be equivalent to excluding rows with the unseen targets from dataset.

Multitreatment split criteria. Generally, according to Equation 1, the tree-building algorithm estimates splits during the tree structure search as the sum of scoring functions obtained separately by the control and treatment groups. However, as shown in [15], this may not be the best strategy for building boosting trees in a multioutput setup. The selection of random outputs for scoring splits performs better in terms of accuracy and speed. This approach also makes sense in terms of estimating causal effects. We can imagine a situation where different features have varying performance between the control and multiple treatment groups. In such cases, a feature that is important only in one of the groups may be rarely selected for building a tree. Therefore, we slightly modified the procedure such that on each boosting stage, we select only a single treatment or control group to build a tree structure, while using all the outputs to estimate leaf values and make a boosting step.

Difference between the components. To visualize the distinction between the **Outcome** and **Uplift** components, we generated synthetic datasets with 2 features and a binary treatment. As depicted in Figure 2, the design ensures that one feature is highly correlated with absolute outcome values, while the other is correlated with the causal effects.

The two-staged gradient boosting model was trained to estimate the causal effect. In Figure 3 and Table 1, we provide a comparison of both model components. It can be observed that the predictions of the **Outcome** and **Uplift** components are highly correlated and closely distributed. Moreover, they show similar predictive power in terms of the R^2 score between the real simulated uplift and the model outputs. However, upon closer examination of how the models make decisions, it becomes apparent that the **Outcome** model relies more heavily on *Feature 0*, while the **Uplift** model more frequently splits by *Feature 1*. This difference is explainable by the data generation process, and it also indicates that the **Uplift** model is more useful for estimating feature importances. Additionally, it can be observed that the **Uplift** model is much simpler in terms of the total number of tree splits in the ensemble. This aligns with our expectations, as the **Uplift** model solves a more straightforward task by estimating only the single surrogate target, rather than multiple outcome values.

Implementation. The proposed method was implemented using the **Py-Boost** library¹. The implementation, along with the code for numerical experiments, is available in the repository².

¹<https://github.com/sb-ai-lab/Py-Boost>

²<https://github.com/sb-ai-lab/uplift-via-gbdt>

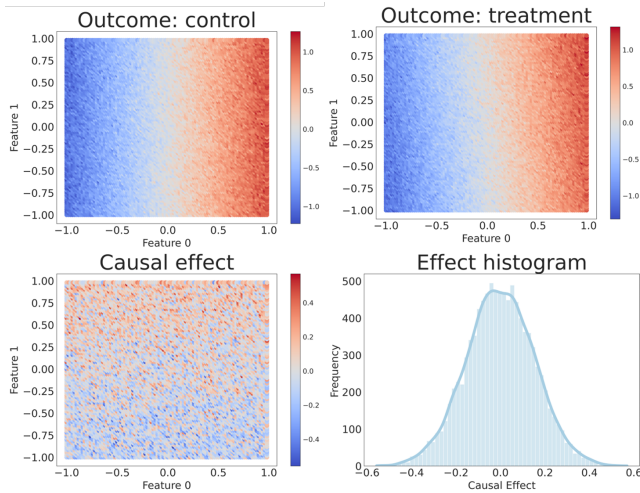


Figure 2: Outcome and causal effect for synthetic data

The design ensures that one feature is highly correlated with absolute outcome values, while another is correlated with the causal effects.

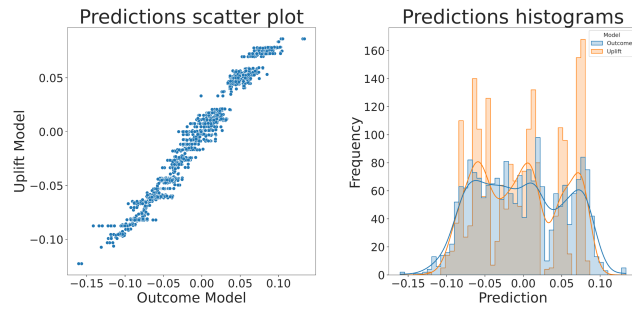


Figure 3: Outcome and Uplift components predictions.

Table 1: Comparison of two staged model components.

Model	R^2	Number of feature splits		
		Total	Feature 0	Feature 1
Outcome	0.1582	2483	1594	889
Uplift	0.1589	1199	299	900
Average	0.1595	3682	1893	1789

4.2 Numerical experiments

In this section, we numerically compare the proposed approach with methods commonly used in practice for uplift modeling and estimating causal effects.

Datasets. The experiments are conducted on four binary outcome real-world datasets from the e-commerce and retail domains, provided in the `scikit-uplift` library³. This benchmark contains three binary treatment datasets: *Criteo*, *Lenta*, and *Megafon*; and one two-treatment dataset: *Hillstrom*. Additionally, we generated a synthetic dataset with six treatments and binary outcomes generated using the `causalml` library⁴. Detailed descriptions of the

datasets are provided in the Appendix Section A.2. For all datasets, the performance metrics were evaluated independently for each treatment.

Metrics. In real-world experiments, it is impossible to directly estimate the accuracy of causal effect prediction since we can only observe the control or one of the treatment results, but not the real effect value. Thus, ranking metrics based on the area under the curve are commonly used in the industry. In our study, for all datasets, we evaluate the AUUC. However, we still aim to accurately predict the effect value, and therefore, we included the synthetic dataset in the analysis. For this dataset, we can evaluate the mean squared error (MSE) between the predicted uplift and the real simulated effect. The overall performance across all datasets/treatments will be summarized as the rank of each algorithm and the percentage difference between the model and the top performer within the task.

Baselines. In our study, we consider the following list of baseline models for comparison, as mentioned in Section 2.3:

- **Meta Learners.** This category includes commonly used *meta learners* that employ a combination of base learner ML models to estimate the uplift value. We include four methods implemented in the `causalml` library: *T-Learner*, *X-Learner*, *R-Learner*, and *DR-Learner*. All methods use `XGBoost`⁵ as a base learner.
- **Causal Forest.** This consists of an ensemble of decision trees directly fitting to estimate the causal effect, implemented in the `EconM`⁶ library.
- **Neural Networks.** This category is represented by the two types of models: `DragonNet`⁷ and `DESCN`⁸.

Our approach. We evaluated the performance of our method in three variations: outcome component only, uplift component only, and the average of the predictions with equal weight. We will refer to them as **GBDT Outcome**, **GBDT Uplift**, and **GBDT Average**, respectively.

Experiment design. We employed a nested cross-validation scheme to obtain the performance results. Each dataset was split into 5 random outer folds, and for each experiment run, one of the outer folds was considered as the test set while the rest served as the training set. Subsequently, the training set was further split into 5 folds to perform the inner cross-validation loop. During this stage, we searched for the best hyperparameters for each baseline based on the inner cross-validation score. We utilized the TPE Search algorithm [23] to maximize the AUUC metric for selecting the best hyperparameter set. Details about the parameter search are provided in the Appendix Section A.3. Afterward, the best-selected baseline configuration was evaluated on the test set. The final model performance metric was calculated as the average value across all outer test folds. In the case of multiple treatments tasks, we trained a single baseline model for all treatments while evaluating the performance independently for each treatment.

Results. The detailed results of the experiments are presented in Tables 3, 4, and 5. As illustrated in Table 5, our method in both

⁵<https://github.com/dmlc/xgboost>

⁶<https://github.com/py-why/EconML>

⁷<https://github.com/farazmah/dragonnet-pytorch>

⁸<https://github.com/kailiang-zhong/DESCN>

³<https://github.com/maks-sh/scikit-uplift>

⁴<https://github.com/uber/causalml>

Reason
for using
synthesis
dataset

Table 2: Summary across all datasets

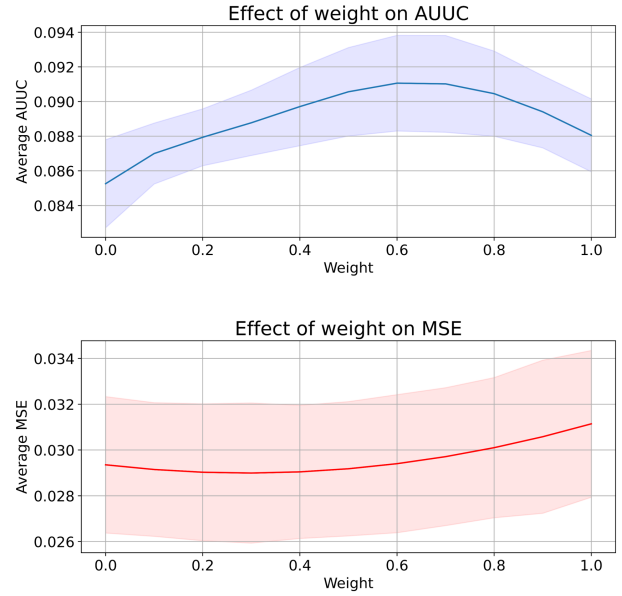
Model	Average Rank		% Loss against top 1	
	AUUC	MSE	AUUC	MSE
T-Learner	4.0	2.3	18.7	4.6
X-Learner	5.4	4.8	20.8	22.9
R-Learner	5.5	6.7	18.5	30.0
DR-Learner	6.3	8.3	20.4	39.7
DragonNet	7.4	6.2	21.6	37.9
DESCN	7.0	6.0	31.7	30.2
CausalForest	7.8	10.0	23.9	127.6
GBDT Outcome	2.5	1.7	10.3	2.0
GBDT Uplift	6.1	6.3	21.7	28.3
GBDT Average	3.2	2.7	7.0	8.9

GBDT Outcome and Average variations demonstrates superior performance on two datasets - *Lenta* and both treatments of *Hillstrom* in terms of the ranking metric AUUC. However, R/DR-learners perform better on *Criteo*, and DragonNet is the optimal choice for *Megafon*. The study on the synthetic dataset reveals that the T-Learner achieves the top AUUC on 4 out of 6 treatments, while our GBDT Outcome and Average results are very close with 3 out of 6. Comparing models by MSE against the simulated effect, we observe that our GBDT Outcome is the top performer, obtaining 4 out of 6 top scores in both cases.

As observed, there isn't a clear winner or universal model across the all tasks. Therefore, we have summarized all the results in Table 2, where we compare the model performance ranks and metric differences. Typically, our GBDT Outcome and Average models achieve higher rankings in the AUUC metric, with overall ranks of 2.5 and 3.2, respectively. Even in cases where GBDTs do not secure the top spot, they are very close, with mean differences of only 7% to 10.3% from the highest possible score. The best competitor for them is the T-Learner with an average rank of 4.0 and a 18.7% mean difference from the leader.

Regarding error against simulated effect, our GBDT Outcome method exhibits the best performance with an overall rank of 1.7 and only a 2% difference with the leader.

One disappointing fact we discovered here is that GBDT Uplift part standalone was unable to achieve the competitive results in any category. But for some of the tasks GBDT Average shows better performance and it is the most stable in terms of % loss against top AUUC in general. That means GBDT Uplift part can help to improve the main GBDT Outcome model. The difficulty in selecting a universal model from the provided baselines highlights the challenge in addressing all tasks effectively. However, one advantage of the two-staged gradient boosting approach is that the model is trained only once, and after that, inference can be performed in any mode by simply adjusting the weights of the Outcome and Uplift components. Figure 4 illustrates how model performance for ranking and effect estimation tasks can vary depending on the weights assigned to these components for one of the treatments of the synthetic datasets. Even if GBDT Outcome usually performs better standalone, for this particular treatment we can select the best weight after model is trained.

**Figure 4: Comparison of model performance depending on components weights for the treatment number 1 of the synthetic dataset**

Discussion on Performance Differences. The performance of GBDT Outcome and GBDT Average varies across different datasets. The primary reason for this variability is that the uplift modeling algorithm may focus on different aspects of the problem depending on the characteristics of the dataset. For some datasets, the algorithm might prioritize accurate outcome predictions, resulting in better performance of the GBDT Outcome model. In contrast, other datasets might have key features that significantly influence the uplift value, making the GBDT Uplift model being helpful to improve the ensemble and useful for interpretation purposes. The GBDT Average model often provides a balanced performance, but its effectiveness can also vary depending on how well the contributions of the Outcome and Uplift components align with the dataset's characteristics. This indicates that the choice of model should consider the specific properties of the dataset and the primary objectives of the uplift modeling task.

5 CONCLUSION

In this study, we have presented a novel approach to uplift modeling that extends the application of Gradient Boosting to multi-treatment models. Our method, which builds upon the existing Gradient Boosting implementation, addresses the challenge of fitting separate models for treatment and control outcome values. Our approach involves a two-staged modeling process. The first stage is a multioutput regression (or classification) Gradient Boosting model that predicts every treatment and control output using the same trees. This uniformity leads to more stable uplift predictions. The second stage involves training a secondary boosting model on a surrogate uplift target derived from the primary model. This secondary model is designed to find simpler surfaces that explain the uplift dependence on features, leading to more robust predictions.

Table 3: AUUC scores for synthetic 6 treatment dataset

Model	Treatment Number					
	1	2	3	4	5	6
T-Learner	0.078±0.0033	0.172±0.0018	0.198±0.0024	0.242±0.0029	0.274±0.0012	0.296±0.002
X-Learner	0.091±0.0019	0.165±0.0018	0.196±0.0016	0.233±0.0018	0.266±0.0007	0.291±0.0006
R-Learner	0.084±0.0028	0.165±0.0038	0.191±0.0014	0.23±0.0015	0.266±0.0025	0.288±0.002
DR-Learner	0.082±0.0034	0.161±0.0013	0.194±0.0011	0.228±0.0013	0.264±0.0016	0.285±0.0016
DragonNet	0.054±0.0032	0.158±0.0025	0.178±0.005	0.2±0.0056	0.251±0.0053	0.266±0.0138
DESCN	0.064±0.0046	0.165±0.0035	0.184±0.0046	0.222±0.0017	0.257±0.0028	0.284±0.0028
CausalForest	0.055±0.0037	0.163±0.0011	0.188±0.0009	0.214±0.0016	0.242±0.002	0.243±0.0015
GBDT Outcome	0.085±0.0025	0.172±0.0037	0.197±0.0024	0.237±0.0013	0.272±0.0013	0.297±0.0018
GBDT Uplift	0.088±0.0021	0.166±0.0021	0.192±0.0015	0.227±0.001	0.264±0.0011	0.288±0.0011
GBDT Average	0.091±0.0026	0.17±0.0027	0.196±0.0035	0.235±0.0017	0.27±0.0012	0.295±0.0016

Table 4: MSE between simulated and predicted effect

Model	Treatment Number					
	1	2	3	4	5	6
T-Learner	0.034±0.003	0.032±0.0028	0.032±0.0025	0.055±0.0035	0.04±0.0025	0.047±0.0031
X-Learner	0.032±0.003	0.036±0.003	0.038±0.0029	0.062±0.0036	0.055±0.0028	0.062±0.0034
R-Learner	0.034±0.0031	0.037±0.003	0.042±0.0029	0.065±0.0035	0.058±0.0028	0.065±0.0033
DR-Learner	0.035±0.0033	0.04±0.0031	0.044±0.0029	0.069±0.0034	0.063±0.0031	0.071±0.0034
DragonNet	0.039±0.0037	0.032±0.003	0.063±0.0035	0.064±0.0043	0.041±0.0033	0.073±0.0037
DESCN	0.045±0.0038	0.035±0.0032	0.038±0.0031	0.068±0.0043	0.051±0.0033	0.062±0.0042
CausalForest	0.05±0.0043	0.059±0.0041	0.079±0.0044	0.106±0.005	0.116±0.0051	0.118±0.005
GBDT Outcome	0.029±0.003	0.029±0.0029	0.031±0.0027	0.055±0.0034	0.042±0.0027	0.049±0.0031
GBDT Uplift	0.031±0.0031	0.036±0.0031	0.04±0.003	0.065±0.0034	0.059±0.0029	0.068±0.0035
GBDT Average	0.029±0.003	0.031±0.003	0.033±0.0027	0.057±0.0032	0.047±0.0025	0.055±0.0031

Table 5: AUUC scores for real-world datasets

Model	Binary Treatment			Hillstrom	
	criteo	lenta	megafon	woman	men
T-Learner	0.045±0.0003	0.005±0.0005	0.166±0.0002	0.002±0.0009	0.024±0.002
X-Learner	0.051±0.0004	0.003±0.0033	0.164±0.0002	0.0±0.0009	0.022±0.0009
R-Learner	0.053±0.0002	0.003±0.0005	0.162±0.0001	0.004±0.0015	0.025±0.0014
DR-Learner	0.053±0.0003	0.003±0.001	0.16±0.0002	0.002±0.0017	0.024±0.0006
DragonNet	0.039±0.003	0.006±0.0034	0.181±0.0003	0.008±0.0016	0.024±0.0015
DESCN	-0.014±0.0061	-0.007±0.0001	0.172±0.0004	0.013±0.0009	0.024±0.0032
CausalForest	0.048±0.0004	0.002±0.0008	0.152±0.0006	0.007±0.0012	0.026±0.0013
GBDT Outcome	0.049±0.0002	0.005±0.0004	0.168±0.0002	0.011±0.0012	0.028±0.0017
GBDT Uplift	0.041±0.0003	0.017±0.0009	0.164±0.0002	0.001±0.0013	0.003±0.0012
GBDT Average	0.045±0.0004	0.013±0.0008	0.168±0.0001	0.01±0.0012	0.028±0.0015

Through comprehensive experimentation on real-world and synthetic datasets, we have demonstrated that our approach surpasses the limitations of current methods in uplift modeling.

REFERENCES

- [1] Susan Athey and Guido Imbens. 2016. Recursive partitioning for heterogeneous causal effects. *Proceedings of the National Academy of Sciences* 113, 27 (2016), 7353–7360.
- [2] Susan Athey and Guido W Imbens. 2015. Machine learning methods for estimating heterogeneous causal effects. *stat* 1050, 5 (2015), 1–26.
- [3] Ismail Babajide Mustapha and Faisal Saeed. 2016. Bioactive molecule prediction using extreme gradient boosting. *Molecules* 21, 8 (2016), 983.
- [4] Olivier Chapelle and Yi Chang. 2011. Yahoo! learning to rank challenge overview. In *Proceedings of the learning to rank challenge*. PMLR, 1–24.
- [5] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [6] Chen Cheng, Fen Xia, Tong Zhang, Irwin King, and Michael R Lyu. 2014. Gradient boosting factorization machines. In *Proceedings of the 8th ACM Conference on Recommender systems*. 265–272.
- [7] Lei Deng, Juan Pan, Xiaojie Xu, Wenyi Yang, Chuyao Liu, and Hui Liu. 2018. PDRLGB: precise DNA-binding residue prediction using a light gradient boosting machine. *BMC bioinformatics* 19, 19 (2018), 135–145.
- [8] Floris Devriendt, Jente Van Belle, Tias Guns, and Wouter Verbeke. 2020. Learning to rank for uplift modeling. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2020), 4888–4904.
- [9] Benoit Frénay and Michel Verleysen. 2013. Classification in the presence of label noise: a survey. *IEEE transactions on neural networks and learning systems* 25, 5 (2013), 845–869.
- [10] Jerome H Friedman. 2001. Greedy function approximation: a gradient boosting machine. *Annals of statistics* (2001), 1189–1232.

- [11] Donald P Green and Holger L Kern. 2012. Modeling heterogeneous treatment effects in survey experiments with Bayesian additive regression trees. *Public opinion quarterly* 76, 3 (2012), 491–511.
- [12] Pierre Gutierrez and Jean-Yves G  rardy. 2017. Causal inference and uplift modelling: A review of the literature. In *International conference on predictive applications and APIs*. PMLR, 1–13.
- [13] Behram Hansotia and Brad Rukstales. 2002. Incremental value modeling. *Journal of Interactive Marketing - J INTERACT MARK* 16 (06 2002), 35–46. <https://doi.org/10.1002/dir.10035>
- [14] Jennifer L Hill. 2011. Bayesian nonparametric modeling for causal inference. *Journal of Computational and Graphical Statistics* 20, 1 (2011), 217–240.
- [15] Leonid Iosipoi and Anton Vakhrushev. 2022. SketchBoost: Fast Gradient Boosted Decision Tree for Multioutput Problems. *Advances in Neural Information Processing Systems* 35 (2022), 25422–25435.
- [16] Shahin Jabbari, Robert C Holte, and Sandra Zilles. 2012. Pac-learning with general class noise models. In *KI 2012: Advances in Artificial Intelligence: 35th Annual German Conference on AI, Saarbr  cken, Germany, September 24-27, 2012. Proceedings* 35. Springer, 73–84.
- [17] Edward H Kennedy. 2020. Towards optimal doubly robust estimation of heterogeneous causal effects. *arXiv preprint arXiv:2004.14497* (2020).
- [18] S  ren R K  nzel, Jasjeet S Sekhon, Peter J Bickel, and Bin Yu. 2019. Metalearners for estimating heterogeneous treatment effects using machine learning. *Proceedings of the national academy of sciences* 116, 10 (2019), 4156–4165.
- [19] Erin LeDell and Sebastien Poirier. 2020. H2o automl: Scalable automatic machine learning. In *Proceedings of the AutoML Workshop at ICML*, Vol. 2020.
- [20] Ping Li, Qiang Wu, and Christopher Burges. 2007. Mcrank: Learning to rank using multiple classification and gradient boosting. *Advances in neural information processing systems* 20 (2007), 897–904.
- [21] Xiaoliang Ling, Weiwei Deng, Chen Gu, Hucheng Zhou, Cui Li, and Feng Sun. 2017. Model ensemble for click prediction in bing search ads. In *Proceedings of the 26th International Conference on World Wide Web Companion*. 689–698.
- [22] Xinkun Nie and Stefan Wager. 2021. Quasi-oracle estimation of heterogeneous treatment effects. *Biometrika* 108, 2 (2021), 299–319.
- [23] Yoshihiko Ozaki, Yuki Tanigaki, Shuhei Watanabe, and Masaki Onishi. 2020. Multiobjective tree-structured parzen estimator for computationally expensive optimization problems. In *Proceedings of the 2020 genetic and evolutionary computation conference*. 533–541.
- [24] Nicholas J Radcliffe and Patrick D Surry. 2011. Real-world uplift modelling with significance-based uplift trees. *White Paper TR-2011-1, Stochastic Solutions* (2011), 1–33.
- [25] Claudia Shi, David Blei, and Victor Veitch. 2019. Adapting neural networks for the estimation of treatment effects. *Advances in neural information processing systems* 32 (2019).
- [26] Xiaogang Su, Joseph Kang, Juanjuan Fan, Richard Levine, and Xin Yan. 2012. Facilitating Score and Causal Inference Trees for Large Observational Data. *Journal of Machine Learning Research* 13 (10 2012), 2955–2994.
- [27] Xiaogang Su, Chih-Ling Tsai, Hansheng Wang, David M. Nickerson, and Bogong Li. 2009. Subgroup Analysis via Recursive Partitioning. *Econometrics: Single Equation Models eJournal* (2009). <https://api.semanticscholar.org/CorpusID:12755477>
- [28] Samir Touzani, Jessica Granderson, and Samuel Fernandes. 2018. Gradient boosting machine for modeling the energy consumption of commercial buildings. *Energy and Buildings* 158 (2018), 1533–1543.
- [29] Ilya Trofimov, Anna Kornetova, and Valery Topinskiy. 2012. Using boosted trees for click-through rate prediction for sponsored search. In *In Proceedings of the Sixth International Workshop on Data Mining for Online Advertising and Internet Economy*. 1–6.
- [30] Stefan Wager and Susan Athey. 2018. Estimation and inference of heterogeneous treatment effects using random forests. *J. Amer. Statist. Assoc.* 113, 523 (2018), 1228–1242.
- [31] Yanru Zhang and Ali Haghani. 2015. A gradient boosting method to improve travel time prediction. *Transportation Research Part C: Emerging Technologies* 58 (2015), 308–324.
- [32] Kailiang Zhong, Fengtong Xiao, Yan Ren, Yaorong Liang, Wenqing Yao, Xiaofeng Yang, and Ling Cen. 2022. Descn: Deep entire space cross networks for individual treatment effect estimation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4612–4620.

A EXPERIMENT DETAILS

A.1 Two-staged gradient boosting algorithm

Algorithm of proposed method is provided as Algorithm 1. We assume, that the treatments vector W consists of integers representing the treatment id and 0 stands for control group. Number

of treatments t is calculated including the control group, i.e. for binary treatment task $t = 2$.

Algorithm 1: Two-staged uplift gradient boosting

```

Data:  $X$ , feature matrix ;
 $Y$ , outcome vector ;
 $W$ , treatment vector ;
 $n$ , number of samples ;
 $t$ , number of treatments ;
 $s$ , number of steps ;
 $treeParams$ , dictionary of tree parameters ;
Result: Tree ensembles  $treesOutcome$  and  $treesUplift$ 
 $treesOutcome = List()$ ;  $treesUplift = List()$ ;
//Initialize  $\hat{Y}$ , size  $(n, t)$ , and  $\hat{U}$ , size  $(n, t - 1)$ 
 $\hat{Y}^0[:, j] = Avg(Y[W = j])$  for  $j$  in  $0..t - 1$ ;
 $\hat{U}^0[:, j - 1] = \hat{Y}^0[:, j] - \hat{Y}^0[:, 0]$  for  $j$  in  $1..t - 1$ ;
for  $b \leftarrow 0$  to  $s - 1$  do
    //Outcome grad and hess, size  $(n, t)$ 
    for  $i \leftarrow 0$  to  $n - 1$  do
        for  $j \leftarrow 0$  to  $t - 1$  do
            if  $W[i] = j$  then
                 $G_o[i, j] = Grad(Y[i], \hat{Y}^b[i, j]);$ 
                 $H_o[i, j] = Hess(Y[i], \hat{Y}^b[i, j]);$ 
            else
                 $G_o[i, j] = H_o[i, j] = 0;$ 
            end
        end
    end
     $tree = BuildTree(X, G_o, H_o)$ ;
     $\hat{Y}^{b+1} = \hat{Y}^b + tree(X)$ ;
     $treesOutcome.append(tree)$ ;
    //Uplift grad and hess, size  $(n, t - 1)$ 
    for  $i \leftarrow 0$  to  $n - 1$  do
        for  $j \leftarrow 1$  to  $t - 1$  do
            if  $W[i] = 0$  or  $W[i] = j$  then
                if  $W[i] = 0$  then
                     $u = \hat{Y}^{b+1}[i, j] - Y[i];$ 
                else
                     $u = Y[i] - \hat{Y}^{b+1}[i, 0];$ 
                end
                 $G_u[i, j - 1] = \hat{U}^b[i, j - 1] - u;$ 
                 $H_u[i, j - 1] = 1$ 
            else
                 $G_u[i, j - 1] = H_u[i, j - 1] = 0;$ 
            end
        end
    end
    end
     $tree = BuildTree(X, G_u, H_u)$ ;
     $\hat{U}^{b+1} = \hat{U}^b + tree(X)$ ;
     $treesUplift.append(tree)$ ;
end

```

Table 6: Summary datasets statistics

Dataset	Rows	Features	Control Outcome	Treatments outcome
hillstrom	64000	8	0.106	[0.182, 0.151]
criteo	1656929	12	0.345	0.404
lenta	687029	193	0.103	0.110
megafon	600000	50	0.179	0.229

A.2 Datasets description

Our experiments were conducted on four real world datasets. Those datasets were obtained via `scikit-uplift` library API. Summary datasets statistics are provided in Table 6.

The original *criteo* dataset contains 14M samples with average control outcome rate 0.038. To speed up computations, we performed the negative sampling and Table 6 is the actual information about the setup used for the experiments.

The synthetic dataset was generated using the function provided in `causalml.dataset.make_uplift_classification`⁹. General parameters to obtain the dataset are:

- `n_samples=10000` (for each treatment)
- `n_classification_features=100`
- `n_classification_informative=20`
- `n_classification_redundant=10`
- `n_classification_repeated=10`
- `positive_class_proportion=0.2`
- `random_state=42`

Treatment specific parameters are provided below:

- `delta_uplift_increase_dict: [0.05, 0.1, 0.12, 0.17, 0.2]`
- `delta_uplift_decrease_dict: [0.01, 0.02, 0.03, 0.05, 0.06, 0.07]`
- `n_uplift_increase_mix_informative_dict: [1, 2, 3, 4, 5, 6]`
- `n_uplift_decrease_mix_informative_dict: [1, 1, 1, 1, 1, 1]`

A.3 Hyperparameters tuning

As it was mentioned in Section 4.2 we perform a nested-cross validation loop to estimate the model performance on the each test fold. During every evaluation, we search for the best hyperparameters based on the cross-validation scores. Thus, each test score was obtained with individual hyperparameters set, that corresponds the best cross-validation results over the train folds.

To search the best hyperparameters we used **TPE Search** algorithm implemented inside **optuna** framework¹⁰. The parameters search setup was designed in following way:

- (1) Each model run was limited in the number of steps. For all tree-based algorithms we limited the number of steps to 500 trees. For DragonNet baseline we limited the number of steps to 30 epochs.

Table 7: Hyperparameters search spaces

Model	Framework	Parameter	Type	Space	Log
Meta Learners	XGBoost	learning_rate	float	[0.005; 0.1]	False
Meta Learners	XGBoost	max_depth	int	[0.7; 1]	False
Meta Learners	XGBoost	subsample	float	[0.7; 1]	False
Meta Learners	XGBoost	colsample	float	[0.7; 1]	False
Meta Learners	XGBoost	lambda	float	[0.1; 50]	True
Meta Learners	XGBoost	min_child_weight	float	[0.001; 10]	True
Meta Learners	XGBoost	gamma	float	[0.001; 100]	True
Causal Forest	EconML	criterion	cat	het/mse	
Causal Forest	EconML	honest	bool	False/True	
Causal Forest	EconML	max_depth	int	[0.7; 1]	False
Causal Forest	EconML	min_samples_leaf	int	[2; 100]	True
Causal Forest	EconML	max_features	float	[0.2; 0.8]	False
Causal Forest	EconML	max_samples	float	[0.1, 0.5]	False
Causal Forest	EconML	min_balancedness_tol	float	[0.05, 0.45]	False
Two-staged GBDT	Py-Boost	learning_rate	float	[0.005; 0.1]	False
Two-staged GBDT	Py-Boost	max_depth	int	[0.7; 1]	False
Two-staged GBDT	Py-Boost	subsample	float	[0.7; 1]	False
Two-staged GBDT	Py-Boost	colsample	float	[0.7; 1]	False
Two-staged GBDT	Py-Boost	lambda_l2	float	[0.1; 50]	True
Two-staged GBDT	Py-Boost	min_data_in_leaf	int	[1; 100]	True
Two-staged GBDT	Py-Boost	min_gain_to_split	float	[0.001; 100]	True
DragonNet	torch	hidden_size	float	[0.5; 2]	False
DragonNet	torch	outcome_size	float	[0.5; 2]	False
DragonNet	torch	alpha	float	[0.5; 1.5]	False
DragonNet	torch	beta	float	[0.5; 1.5]	False
DragonNet	torch	steps_per_epoch	int	[100; 300]	True
DragonNet	torch	learning_rate	int	[0.0001; 0.001]	False
DESCN	torch	share_scale	float	[0.5; 2]	False
DESCN	torch	base_scale	float	[0.5; 2]	False
DESCN	torch	prpsy_w	float	[0.5; 1]	False
DESCN	torch	escvr1_w	float	[0.5; 1]	False
DESCN	torch	escvr0_w	float	[0.5; 1]	False
DESCN	torch	mu0hat_w	float	[0.5; 1]	False
DESCN	torch	mu1hat_w	float	[0.5; 1]	False
DESCN	torch	steps_per_epoch	int	[100; 300]	False
DESCN	torch	learning_rate	int	[0.0001; 0.001]	False

- (2) Each TPE Search run was limited by 50 steps and 10000 seconds. Every TPE Search run cross-validation loop over the training set was performed.

- (3) At the each step we evaluate the set of sampled hyperparameters. Sampling parameters for each framework are provided in Table 7

For the DragonNet baseline, the `hidden_size` parameter provided represents the size of the hidden layer as a multiplier on the number of features. Similarly, the `outcome_size` parameter refers to the multiplier on the hidden layer size to obtain the outcome layer size. The same rule is used for DESCN parameters `share_scale` and `base_scale`.

⁹causalml library <https://github.com/uber/causalml/dataset/classification.py>

¹⁰<https://github.com/optuna/optuna>